

JavaScript API Web Workers

[Back to JS page](#)

Note: For a complete tutorial on creating and managing Web Workers, see [05.2065-js-WebWorkers](#). This topic covers the Web Workers API from the Web API perspective.

Table of Contents

- [JavaScript API Web Workers](#)
 - [Web Workers Example](#)
 - [Example 1](#)
 - [Terminate a Web Worker](#)
 - [Example 2](#)
 - [Web Workers and the DOM](#)
 - [Example 3](#)
 - [Document](#)
 - [Reference](#)

What is a Web Worker?

A Web Worker is a JavaScript running in the background, without affecting the performance of the page. While the worker runs in the background, you can continue doing whatever you want: clicking, selecting things, etc.

When a worker runs, the code runs independently in a separate thread.

Web Workers Example

The example below creates a web worker that counts numbers in the background:

worker.js - The worker file:

```
let i = 0;
function timedCount() {
  i++;
  postMessage(i);
  setTimeout("timedCount()", 500);
}
timedCount();
```

Main page:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript API Web Workers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Web Workers Example</h4>

<p>Count numbers: <output id="result"></output></p>
<button onclick="startWorker()">Start Worker</button>
<button onclick="stopWorker()">Stop Worker</button>

<p id="status"></p>

<script>
let w;

function startWorker() {
  if (typeof(Worker) !== "undefined") {
    if (typeof(w) === "undefined") {
      w = new Worker("worker.js");
    }
    w.onmessage = function(event) {
      document.getElementById("result").innerHTML = event.data;
    };
    document.getElementById("status").innerHTML = "Worker running...";
  } else {
    document.getElementById("status").innerHTML = "Web Workers not supported.";
  }
}

function stopWorker() {
  if (w) {
    w.terminate();
    w = undefined;
    document.getElementById("status").innerHTML = "Worker stopped.";
  }
}
</script>

</body>
</html>

```

Example 1

Result [View Example](#)

Terminate a Web Worker

When a web worker is created, it continues to run (even after the page is closed) until it is terminated. Use `terminate()` to stop it:

worker.js - Worker that counts continuously:

```
let counter = 0;
function count() {
  counter++;
  postMessage(counter);
  setTimeout(count, 1000);
}
count();
```

Main page:

```

<!DOCTYPE html>
<html>
<body>

<h2>JavaScript API Web Workers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Terminate a Web Worker</h4>

<p>Timer: <span id="timer">0</span> seconds</p>
<button onclick="startTimer()">Start Timer</button>
<button onclick="stopTimer()">Stop Timer</button>
<p id="status2"></p>

<script>
let timerWorker;

function startTimer() {
  if (typeof(Worker) !== "undefined") {
    if (typeof(timerWorker) === "undefined") {
      timerWorker = new Worker("timer_worker.js");
    }
    timerWorker.onmessage = function(event) {
      document.getElementById("timer").innerHTML = event.data;
    };
    document.getElementById("status2").innerHTML = "Timer running in worker...";
  } else {
    document.getElementById("status2").innerHTML = "Web Workers not supported.";
  }
}

function stopTimer() {
  if (timerWorker) {
    timerWorker.terminate();
    timerWorker = undefined;
    document.getElementById("status2").innerHTML = "Timer stopped with terminate().";
  }
}
</script>

</body>
</html>

```

Example 2

Result [View Example](#)

Web Workers and the DOM

Since web workers are in external files, they do NOT have access to:

- The DOM (document object)
- The window object
- The parent object

They CAN access:

- The `navigator` object
- The `location` object (read-only)
- `setTimeout()`, `setInterval()`, `clearTimeout()`, `clearInterval()`
- The `XMLHttpRequest` object (for AJAX calls)

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript API Web Workers</h2>
<p>From w3schools.com, Experiment by Teeratus_R</p>

<h4>Web Workers and the DOM</h4>
<p id="demo3"></p>

<script>
let text = "Web Worker Restrictions:<br><br>";

text += "Workers CANNOT access:<br>";
text += "x The DOM (document object)<br>";
text += "x The window object<br>";
text += "x The parent object<br><br>";

text += "Workers CAN access:<br>";
text += "✓ navigator object<br>";
text += "✓ location object (read-only)<br>";
text += "✓ setTimeout()/setInterval()<br>";
text += "✓ XMLHttpRequest (AJAX)<br><br>";

text += "Communication happens via:<br>";
text += "worker.postMessage() - send data to worker<br>";
text += "worker.onmessage - receive data from worker<br>";
text += "worker.terminate() - stop the worker";

document.getElementById("demo3").innerHTML = text;
</script>

</body>
</html>
```

Example 3

Result [View Example](#)

Document

Document in project

You can [Download PDF](#) file.

Reference

- [W3Schools JavaScript API Web Workers](#)
- See also: [05.2065-js-WebWorkers](#) for detailed examples